Advanced Programming

Final Lecture

Troels Henriksen

Real World Haskell

How to Live the Rest of Your Life

Course Evaluation

The Exam

Leaving the ivory tower

AP	▪ Simple models. ▪ Clear specifications. ▪ No performance concerns.
Real world	▪ Arbitrary complexity. ▪ Unclear requirements. ▪ Performance requirements.

The ivory tower still exists in the real world

- My research is on parallel programming, compiler optimisations, high performance computing, numerical computing, etc.
- **Applied computer science**—I read and write many programs.
  - Haskell is my *pragmatic* tool of choice.

The ivory tower still exists in the real world

- My research is on parallel programming, compiler optimisations, high performance computing, numerical computing, etc.
- **Applied computer science**—I read and write many programs.
  - Haskell is my *pragmatic* tool of choice.

- Main applied project is *Futhark*.
- Over 100k lines of Haskell.
- Haskell is *not the point*—feels like any other program.



`https://futhark-lang.org`

Artwork by Robert Schenck.

So what is it like to use Haskell for real programs?

`String` considered harmful

```
type String = [Char]

"DIKU" == 'D' : 'I' : 'K' : 'U' : []
```

`String` considered harmful

```
type String = [Char]
```

```
"DIKU" == 'D' : 'I' : 'K' : 'U' : []
```

But consider the memory usage.

- Each `:` (*cons*) cell contains two slots: *value* and *successor*.
  - Each is a pointer to somewhere in memory.

`String` considered harmful

```
type String = [Char]
```

```
"DIKU" == 'D' : 'I' : 'K' : 'U' : []
```

But consider the memory usage.

- Each `:` (*cons*) cell contains two slots: *value* and *successor*.
  - Each is a pointer to somewhere in memory.
- A four-character string needs four cells, meaning eight pointers in total.
  - That is $8 \cdot 8 = 64$ bytes!
    - For a four-character string!
    - This is 16*x* overhead.
  - And in fact, each Haskell value also needs a *header* for garbage collection purposes, so things are even worse!

`String` considered harmful

```
type String = [Char]

"DIKU" == 'D' : 'I' : 'K' : 'U' : []
```

But consider the memory usage.

- Each `:` (*cons*) cell contains two slots: *value* and *successor*.
  - Each is a pointer to somewhere in memory.
- A four-character string needs four cells, meaning eight pointers in total.
  - That is $8 \cdot 8 = 64$ bytes!
    - For a four-character string!
    - This is 16*x* overhead.
  - And in fact, each Haskell value also needs a *header* for garbage collection purposes, so things are even worse!
- Also the characters are scattered all over memory $\Rightarrow$ terrible locality.

Real Haskell programs use `Data.Text.Text`

`https://hackage.haskell.org/package/text-2.1.1/docs/Data-Text.html`

- Built on top of `Data.ByteString`.

Haskell cheat code

```
unsafePerformIO ::  IO a -> a
```

- **Extremely Dangerous**, much more than you think.

Every week of AP is useful to me

...but there are reasons to diverge from doctrine.

Real World Haskell

How to Live the Rest of Your Life

Course Evaluation

The Exam

If you liked AP

Unfortunately there is no *Really Advanced Programming* course at DIKU.

But there are courses that have the same spirit.

If you liked AP

Unfortunately there is no *Really Advanced Programming* course at DIKU.

But there are courses that have the same spirit.

- *Programming Massively Parallel Hardware* (PMPH), block 1, taught by Cosmin Oancea. GPU programming. Only course at DIKU where you are specifically graded on writing fast programs.

If you liked AP

Unfortunately there is no *Really Advanced Programming* course at DIKU.

But there are courses that have the same spirit.

- *Programming Massively Parallel Hardware* (PMPH), block 1, taught by Cosmin Oancea. GPU programming. Only course at DIKU where you are specifically graded on writing fast programs.
- *Data Parallel Programming* (DPP), block 2, taught by Cosmin Oancea and myself. High level functional programming and performance. Strongly based on local research.

If you liked AP

Unfortunately there is no *Really Advanced Programming* course at DIKU.

But there are courses that have the same spirit.

- *Programming Massively Parallel Hardware* (PMPH), block 1, taught by Cosmin Oancea. GPU programming. Only course at DIKU where you are specifically graded on writing fast programs.
- *Data Parallel Programming* (DPP), block 2, taught by Cosmin Oancea and myself. High level functional programming and performance. Strongly based on local research.
- *Semantics and Types* (SaT), block 3, taught by Andrzej Filinski. Programming language theory and semantics. **Theoretical.**
- *Interactive Proof Assistants* (IPA), block 1, taught by Dmitriy Traytel. Even richer forms of PL-like modeling.

Also consider projects

I work in the *Programming Languages and Theory of Computation* section, and there are many researchers willing to supervise projects in functional programming and other PL topics.

- 30 ECTS master thesis.
- 7.5 ECTS thesis preparation project.
- 7.5 ECTS or 15 ECTS “project outside course scope”.

Potential supervisors:

- Troels Henriksen
- Martin Elsman
- Torben Mogensen
- Ken Friis Larsen
- Andrzej Filinski
- Cosmin Oancea
- The others: `https://di.ku.dk/english/research/pltc/`

Real World Haskell

How to Live the Rest of Your Life

Course Evaluation

The Exam

Preliminary context

- **The MSc at DIKU is supposed to be challenging.**
  - *Intended* to be the highest academic level of computer science in Denmark.

Preliminary context

- **The MSc at DIKU is supposed to be challenging.**
  - *Intended* to be the highest academic level of computer science in Denmark.
- **AP is supposed to be hard.**
  - Most ambitious *semi-mandatory* programming course in Denmark (that I could find).

Preliminary context

- **The MSc at DIKU is supposed to be challenging.**
  - *Intended* to be the highest academic level of computer science in Denmark.
- **AP is supposed to be hard.**
  - Most ambitious *semi-mandatory* programming course in Denmark (that I could find).
- **Suffering is not the same as learning.**
  - Students don't learn more when they are stressed.
  - Teachers don't benefit from tormenting students.

Looking at graphs

Selected comments

- Some of these have been rephrased, paraphrased, or translated from Danish.
- I consider them all *constructive* and *useful*, even if I do not always agree.

It also seems completely insane that the exam format is being changed so late that there isn't enough time to plan the course for the new format.

Having a clearer picture of what to expect from the exam would have been helpful.

I find it hard to know what to prepare doing the exam

Uncomfortable with the new exam format

Assignments 4 and 6 are much larger workload than the other assignments partially due to containing design tasks for fairly complex problems, and also being the first real design tasks. I would suggest incorporating more design tasks earlier and on simpler problems so that it can be practiced more smoothly.

In assignment 4 the jump in complexity from the exercises to the assignment is much larger than in other assignments.

I found it difficult to find time for both exercises and assignments some weeks.

Within the assignments, I find the heavy dependence of the tasks difficult to manage in small groups. As the assignment group members have different levels of experience with different schedules to meet it's difficult to arrange a study session.

The updated version of the More Monads (week 4) section of the course notes needs to be revisited as it included way too much fluff, which was not directly linked to the exercise and assignments. This part of the course notes needs to be rewritten more concisely. It was the first time that I generally got lost in the material.

I thought the notes for weeks 5 and 6 were a little convoluted, and that they contained a great deal of information that was not particularly coherent.

Write precisely what students are supposed to read, rather than "Suggested Reading".

Even though the course notes are well-condensed relating to the required information for the course, it can be difficult to get an overview of what is the most important, what is less important, and what is optional.

Some of the course notes had changes midweek, which was quite disrupting.

It works really well to keep everything on GitHub.

Fantastic. Absalon is bad. The course's use of GitHub and Discord solves all the problems.

GitHub is set up in an organized way.

The GitHub worked very well.

Actually, the GitHub makes a lot of sense. Continue with that!

It would be helpful to keep the course material in Absalon well-organised, with lecture slides, exercise sheets, assignment instructions and relevant resources grouped by week or topic. Having all important information in one place, along with timely announcements about deadlines and any changes, would make it easier for students to follow the course and plan their work effectively.

In person coding (listed as a good thing)

Also, live coding is at times too quick, even though it gives a nice practical experience, we would also like to see a stronger relation between the practical and theoretical side.

Live coding can be the best way to learn the practices of the theoretical material.

Don't stop live coding, just make sure that everyone can follow along.

We have been repeatedly told that this isn't a haskell course, but so far i have yet to see any evidence supporting this.

*Tell about more examples of the patterns occurring in *other* languages than haskell.*

I would have appreciated it if the course had focused more on why we do certain things and what the actual benefit is. Information about where these techniques are used and when they are required was also missing. Often it felt more like a Haskell course than an AP course.

Instead of Haskell, I would have preferred a language like C.

It is not the task of the University to offer what society asks for, but to give what society needs.

— Edsger Dijkstra, EWD1305 (2000)

Real World Haskell

How to Live the Rest of Your Life

Course Evaluation

The Exam

Written theoretical exam

An exam is a measurement that determines to which extent a student has achieved the intended learning goals.

AP learning goals

Knowledge of

- Higher-level program structuring patterns for separating concerns.
- The basics of message-passing concurrency, and of how concurrent programming can be used for parallel programming.
- Program structuring principles and design patterns for dealing with software errors.
- Property-based testing of functions and stateful APIs

Skills to

- Use program structuring principles and design patterns, such as monads, to structure the code and its potential effects such that there is a clear separation of concerns.
- Use a parser combinator library to write a parser for a medium-sized language with a given grammar, including changing the grammar so that it is on an appropriate form.
- Implement simple concurrent/distributed servers using message passing, with appropriate use of synchronous and asynchronous message passing.
- Use program structuring principles and design patterns for making reliable distributed systems in the presence of software errors.
- Construct systematic test suites for programs and modules, including property-based tests where relevant.

Competences to

- Use advanced programming techniques to solve challenging, realistic problems.
- Write correct, efficient, and maintainable programs with clear separation of concerns.
- Give an assessment of their own code, based on a systematic evaluation of correctness, selection of algorithms and data structures, error scenarios, and elegance.

Performing the measurement

In principle

- No set exam, student *does something* that demonstrates learning, and teacher evaluates.
- **Too much noise, much too difficult to assess.**

Performing the measurement

In principle

- No set exam, student *does something* that demonstrates learning, and teacher evaluates.
- **Too much noise, much too difficult to assess.**

Ideally

- Students shouldn't worry about learning goals or the mechanics of exam grading.
- Just do what is natural based on course contents, e.g. solve some given problem.
- **Now it is the teacher's job to specify a problem that covers all learning goals.**

What to keep in mind for a written exam

1. Be concrete and specific.
2. You are judged on what you write that is correct, but also what you write that is incorrect.
3. Ideally also be concise, but you are not judged on style.
4. Breadth *in learning goals* is important, and more important than depth in a single topic.
5. If something in the text is wrong or unclear, state your assumptions and answer based on those.
6. The grader has access to your exam submission *but not your assignment solutions*.

See you next year

Questions?